

講義 28：客製 Table Maintenance 的權限防護與並行控制（授課順序：接在講義 27 之後）

對應練習：ex28 | 答案物件：表 ZTR28_WPARAM + 權限物件 ZTR28_WERKS + Lock Object EZTR28_WERKS
+ T-code ZTR28_MAINT + 程式 ZR_TR28_PARAM_MAINT / ZR_TR28_PARAM_LIST

本講重點

- 為什麼 SM30 原生的權限檢查不夠用——業務維度（如「只能改自己工廠」）要自己包一層
- 自訂權限物件（Authorization Object）：ACTVT + 業務欄位（本例 WERKS）
- Lock Object 的鎖定範圍可以比表格主鍵更粗——鎖「一個工廠」而不是「一筆資料」
- VIEW_MAINTENANCE_CALL：從程式呼叫標準 Table Maintenance 的正規做法
- T-code 應該指到「包裝程式」，不是直接指到裸的維護畫面
- Report 主程式的 App bar 按鈕怎麼呼叫另一個 Transaction：SET PF-STATUS + AT USER-COMMAND + CALL TRANSACTION

1. 為什麼要包一層 Authorization Wrapper

講義 21 學過：SM30 產生的維護畫面背後靠 Table Maintenance Generator，存取權限預設是靠 S_TABU_DIS（Table Authorization Group）/ S_TABU_NAM（依表名）這兩個通用權限物件控管——它們回答的問題是「這個使用者能不能維護這張表」，是表格層級的粗粒度控管。

實務上常常需要更細的控管：「王小明只能改工廠 1011 的參數，李小美只能改工廠 2011 的參數」——這種業務維度的權限，SM30 原生機制做不到（除非另外設定進階的 Organizational Criteria，那是更複雜的機制，本課不涉及）。

解法：不要把 T-code 直接指給 SM30，而是自己寫一支「包裝程式（Wrapper）」，T-code 指給這支程式。使用者要維護資料，一定先經過這支程式，程式裡先做：

1. 權限檢查（這個使用者對「這個工廠」有沒有維護權限）
2. 上鎖（避免同一個工廠同時有兩個人在維護，造成第 27 講學過的遺失更新）

兩關都過了，才呼叫標準機制把使用者帶進真正的維護畫面。這是企業導入客製主檔維護時的標準模式，值得完整走一次。

2. 自訂權限物件：SU21

1. SU21 → 選一個 Object Class（本例用 BC，Basis 相關；實務上依表格所屬模組選對應 Class）→ Create Authorization Object
2. 物件名稱 ZTR28_WERKS（Z 開頭，10 碼以內）
3. Authorization Fields：加兩個欄位——
 - ACTVT（標準欄位，Activity，代表「做什麼」：01=Create、02=Change、03=Display.....SAP 標準的活動代碼清單）
 - WERKS（本例的業務欄位，代表「對誰」：哪個工廠）
4. 存檔、Activate

ACTVT 幾乎是所有自訂權限物件的標配：光看「使用者對某張表有沒有權限」不夠，還要分「只能看」還是「可以改」。標準活動代碼 01/02/03/06 (刪除) / 08 (顯示變更文件)...可以查 SU21 或 SE11 顯示 Data Element **ACTVT** 的固定值清單。

程式裡用 **AUTHORITY-CHECK** 呼叫這個物件：

```
AUTHORITY-CHECK OBJECT 'ZTR28_WERKS'
  ID 'ACTVT' FIELD lv_actvt
  ID 'WERKS' FIELD p_werks.

IF sy-subrc <> 0.
  " 沒有權限—sy-subrc 常見值：4=沒有這個值的權限、12=系統裡根本沒有這個權限物件
ENDIF.
```

要讓某個使用者真的擁有這個權限，還要在 **PFCG** (角色維護) 把這個權限物件加進角色、給值 (例如 **ACTVT=02**、**WERKS=1011**)，再把角色指派給使用者——這部分是權限管理員的工作，不是 ABAP 開發的範圍，本課不深入，但要知道「權限物件只是定義了『有哪些欄位可以管控』，真正『誰對什麼值有權限』要靠角色維護」。

3. Lock Object 的鎖定範圍可以比主鍵更粗

講義 27 的 **EZTR21_STUD** 鎖 **MANDT+ID** (表格完整主鍵)，意思是「鎖一筆學生資料」。本題的表 **ZTR28_WPARAM** 主鍵是 **MANDT+WERKS+PARAM** (三個欄位)，但 Lock Object **EZTR28_WERKS** 的 Lock Parameters 只勾 **MANDT+WERKS**——刻意不勾 **PARAM**。

為什麼：業務需求是「同一個工廠不能有兩人同時維護」，不是「同一個參數列不能有兩人同時改」——如果鎖到 **PARAM** 這麼細，兩個人分別改同工廠的不同參數列會被允許同時進行，但兩人事實上都在同一個 SM30 畫面 (同一個工廠的整批參數列表) 裡操作，容易互相覆蓋彼此看到的畫面快照。**鎖定範圍要對應到「使用者實際上在爭搶的資源邊界」，不是機械式地照抄表格主鍵**——這是 Lock Object 設計上最容易被忽略的一個判斷。

SE11 建立步驟 (跟講義 27 相同流程，範圍不同)：

1. SE11 → Lock Object → **EZTR28_WERKS** → Create
2. Primary Table：**ZTR28_WPARAM**
3. Lock Parameters：只勾 **MANDT**、**WERKS** (不勾 **PARAM**)
4. Lock Mode：E
5. 存檔、Activate

4. **VIEW_MAINTENANCE_CALL**：從程式呼叫標準 Table Maintenance

土法煉鋼的做法是在程式裡 **CALL TRANSACTION 'SM30'**，但這樣沒辦法乾淨地指定要維護哪個 View、也繞不過 SM30 自己的初始畫面。SAP 提供了正規的函式模組：

```
CALL FUNCTION 'VIEW_MAINTENANCE_CALL'
  EXPORTING
    action      = 'U'          " U=維護 (Update)、S=只顯示 (Show)、T=帶Transport單維護
    view_name   = 'ZTR28_WPARAM'
```

```

EXCEPTIONS
  client_reference          = 1
  foreign_lock              = 2
  invalid_action            = 3
  no_clientindependent_auth = 4
  system_failure            = 5
  OTHERS                    = 6.

```

這支 FM 內部會再做一次它自己的權限檢查 (`S_TABU_DIS/S_TABU_NAM`) 跟再上一次它自己的鎖 (`VIEW_ENQUEUE` , 鎖的是整個 View , 不是我們自訂的 Lock Object) ——這是完全獨立、疊加在我們自訂 Wrapper 之上的第二層保護, 兩層互不衝突: 我們的 Wrapper 負責業務維度 (工廠) , `VIEW_MAINTENANCE_CALL` 自己負責表格層級的通用維護權限與鎖定。

`action = 'S'` (只顯示) 搭配 `ACTVT = '03'` 的權限檢查, 可以做出一個「只能看、不能改」的模式, 這正是本題 Wrapper 程式 `p_disp` 核取方塊要做的事。

5. T-code 要指給 Wrapper , 不是指給裸的 SM30

SE93 建 T-code (做法跟講義 12 之後 Enhancement 課程提過的一致: Program and selection screen 型態) :

- Program 填 `ZR_TR28_PARAM_MAINT` (我們的 Wrapper 程式, 不是 SM30、也不是直接填 View 名稱)
- Screen 填 `1000` (一般報表的選取畫面慣例, 見 Enhancement 課程 en02 的說明)

這是整個模式能不能生效的關鍵一步: 如果貪方便, 直接開一個 T-code 指給 SM30、或是把 `S_TABU_DIS/S_TABU_NAM` 開放給所有人, 使用者就能繞過我們寫的 Wrapper 直接進 SM30 維護, 前面做的權限檢查與 Lock Object 就完全形同虛設。實務上要搭配: 一般使用者的角色不給 **SM30 / 裸 View** 維護的權限, 只給這支 **Wrapper** 的 **T-code**, 才能確保大家只能走這條有檢查的路。

6. Report 主程式的 App bar 按鈕: SET PF-STATUS + AT USER-COMMAND

Classical List Report (`WRITE` 清單) 可以在畫面上方工具列 (App bar) 加自訂按鈕, 透過 **SE41 Menu Painter** 設計, 跟 ALV 用 `IT_EVENTS`/GUI Status 觸發互動 (講義 9 進階篇) 是不同的機制:

1. **SE41** → GUI Status 名稱 (本例 `ZTR28LIST`, 程式名稱填 `ZR_TR28_PARAM_LIST`) → Create
2. Application Toolbar 區塊, 拉一個新按鈕, Function Code 填 `MAINT`、Icon/文字自訂 (如「維護」)
3. 存檔、Activate (Menu Painter 物件本身沒有語法檢查, 存檔啟用即可)

程式裡對應:

```

TOP-OF-PAGE.
  SET PF-STATUS 'ZTR28LIST'.      " 掛上剛剛在 SE41 設計的畫面狀態

AT USER-COMMAND.
  CASE sy-ucomm.                  " 使用者按下的按鈕 Function Code
    WHEN 'MAINT'.
      CALL TRANSACTION 'ZTR28_MAINT'.  " 呼叫 T-code, 不是直接呼叫程式
    ENDCASE.

```

用 **CALL TRANSACTION** 呼叫 **T-code**，而不是 **SUBMIT** 呼叫程式本身：差別在於 **CALL TRANSACTION** 會連 T-code 層級的 **S_TCODE** 權限也一併檢查，**SUBMIT** 則會跳過這一層，直接執行程式——既然我們花心思設計了「只給 Wrapper 的 T-code 授權」這道防線，呼叫方式也要選擇會觸發這道防線的 **CALL TRANSACTION**，才能讓整套設計前後一致。

7. 整體流程總結



四層防護 (T-code 權限、業務權限、Lock Object、SM30 原生防護) 疊在一起，缺任何一層都可能被繞過——這正是企業客製常見「看似簡單的維護畫面，其實包了好幾層防護」的真實樣貌。